

Phase 2 — The Harvest Social Dilemma

What we are building, what each file does, and what we are actually trying to prove

Project: Emergent Social Behaviour & Dilemmas in Multi-Agent Reinforcement Learning | **Student:** Ahmed Wael Elsis (214647) | **Supervisor:** Dr. Randa Mohamed | British University in Egypt.

1. The one-sentence point of the whole project

We want to know **when and why artificial agents learn to cooperate versus exploit each other**. To answer that, we study the same algorithms across a **spectrum** of problems — from one where cooperation is automatic, to one where it is genuinely tempting to cheat.

Phase 1 (Traffic, the RP-5 folder) is the easy end of the spectrum: a 2×2 grid of traffic lights. There is no temptation to defect — if every intersection does its job, everyone wins. We already finished this and found that a coordinated algorithm (MAPPO) and an independent one (IPPO) perform *essentially the same*. In other words, **coordination machinery buys you almost nothing when incentives are already aligned**.

Phase 2 (Harvest, the RP-6 folder — this document) is the hard end: a shared field of apples where grabbing everything for yourself is individually rational but collectively ruinous. This is a genuine **social dilemma**. Here we expect the algorithms and reward structures to *diverge* — and that divergence is the core scientific contribution of the thesis.

So the two phases together form a controlled contrast: **same agents, same learning algorithms, opposite incentive structures**. Phase 1 is the baseline that makes the Phase 2 result meaningful.

2. What Harvest is, and why it is a dilemma

Harvest (Leibo et al., 2017) is a grid world. In our setup it is a **12×8 grid with 4 agents**. Apples are scattered on the grid. Each step, every agent picks one of **6 actions**: move North/South/East/West, stay put, or collect the apple under it. Collecting an apple gives a reward of +1. Episodes last **1000 steps**.

The twist — and the entire reason this is a dilemma — is the **apple regrowth rule**:

$P(\text{an empty cell regrows an apple}) = 0.01 \times (\text{number of apples already nearby, within radius 2})$

Apples only grow back near *other* apples. If a patch is harvested down to zero, **it can never recover for the rest of the episode**. This is a **tragedy of the commons** in miniature:

- **Short-term individual incentive:** grab every apple you can see, now.
- **Long-term collective need:** leave some apples so the patch keeps regenerating, giving everyone more apples over the whole episode.
- **The conflict:** an agent that restrains itself only loses out if the others grab the apples anyway. Greedy behaviour is individually safe but, if everyone does it, the field collapses and everybody ends up poor.

A good outcome looks like **sustainable harvesting** — agents thin the field without destroying it, so apples keep regrowing and the *total* collected over 1000 steps is high. A bad outcome is an early **depletion**: a burst of apples in the first ~100 steps, then a dead field for the remaining 900.

3. What we are actually trying to prove

Phase 2 isolates the thesis's first and central research question:

RQ1 — How does the reward structure (individual vs. shared) change whether cooperation emerges?

We do this by turning a single dial, the **reward-sharing weight** w , which controls how much each agent is paid for its own apples versus the team's average:

- **$w = 0.0$ (individual):** you keep exactly what you collect. This is the canonical social dilemma — defection (grabbing) pays off personally.
- **$w = 0.5$ (mixed):** half your reward is your own apples, half is the team average.
- **$w = 1.0$ (team):** everyone is paid the team average, so hoarding gives you no personal advantage — cooperation is structurally enforced by the reward.

The hypothesis: as w rises from 0 to 1, agents should shift from exploitation toward sustainable harvesting — higher total apples, a healthier field, and fairer (more equal) outcomes. Showing this cleanly answers RQ1.

The second contrast: MAPPO vs. IPPO (the algorithm axis).

On top of the reward dial, we run two learning algorithms that differ in exactly one philosophical way:

- **MAPPO** — "cooperative" training. During learning it uses a *centralized critic* that sees the whole grid (all agents at once), so it can reason about joint outcomes. (This is the CTDE paradigm: centralized training, decentralized execution.)
- **IPPO** — "independent" learning. Each agent has its own critic that sees only its own local view. Each agent treats the others as part of the environment.

Comparing them measures the **value of coordination machinery in a dilemma**. In Phase 1 (traffic) this value was ~zero. The Phase-2 expectation is that in a true dilemma the centralized view *should* help — or, just as interestingly, that reward structure (the w dial) matters far more than the algorithm. Either way, the comparison is the result.

The third piece: a FixedGreedy baseline. A hand-written rule that always walks to and grabs the nearest visible apple. It represents "pure, mindless exploitation" and anchors the scale — it should deplete the field fastest. Any learned policy that beats it has learned some restraint.

4. The experiment matrix (33 runs)

We cross the two axes (3 reward weights $\times$ 2 algorithms) and repeat each with 5 random seeds (42–46) for statistical reliability, plus 3 FixedGreedy runs:

Algorithm	Reward sharing	Seeds	Runs
MAPPO (centralized critic)	individual / mixed / team	5 each	15
IPPO (independent critics)	individual / mixed / team	5 each	15
FixedGreedy (hand-written rule)	n/a	3	3
		TOTAL	33

Each learned run trains for **200 iterations** (~3 hours on the single GPU), so the 30 RL runs take roughly **3.5–4 days** in total, run sequentially. Results are compared pairwise with **Welch's t-test** on the four key

metrics below.

5. How we measure cooperation vs. exploitation

Four numbers per run tell the story:

- **Total apples collected** — collective welfare / efficiency. Higher is better and (in a regrowing field) is itself evidence of restraint.
- **Sustainability** — how many apples remain on the grid late in the episode (fraction of the initial amount, measured over the last 25%). High = the field survived; near-zero = tragedy of the commons.
- **Time-to-depletion** — the step at which the field first hits zero apples (or 1000 if it never does). Late or never = sustainable; early = exploited.
- **Gini coefficient** — equity. 0 = every agent collected an equal share; near 1 = one agent hoarded everything. Answers RQ4 (can high performance coexist with fairness?).

6. What each file in RP-6 represents

The environment (the simulated world)

File	What it represents
marl_env/harvest_env.py	The Harvest world itself. Holds the apple grid and agent positions, applies the 6 actions, runs the regrowth rule each step, hands out rewards, and ends the episode at 1000 steps. Also exposes the full grid as a "global_state" so MAPPO's centralized critic can see everything.
marl_env/harvest_obs.py	Builds what each agent actually sees: a 15×15×3 colour image centred on that agent (black = empty, green = apple, blue = agent, red = wall). Agents see only their local patch, not the whole field.
marl_env/harvest_reward.py	Implements the reward dial. Takes each agent's apple count and blends it with the team average using the sharing weight w (0 / 0.5 / 1.0). This single file is the load-bearing experimental knob for RQ1.
marl_env/harvest_metrics.py	Computes the four scoring numbers (total apples, sustainability, time-to-depletion, Gini/equality) per episode and writes them to CSV in a format shared with Phase 1 so the two phases can be compared side by side.

The brains (neural networks)

File	What it represents
models/mappo_cnn_model.py	MAPPO's network. A small CNN "actor" that picks actions from the local 15×15 view, plus a <i>centralized critic</i> CNN that reads the whole grid to judge how good a situation is. Contains the callback that feeds the global grid into the critic during training (the part that was broken and is now fixed).
models/ippo_cnn_model.py	IPPO's network. Same actor, but its critic sees only the agent's own local view — no global grid. This is the "independent learner" baseline.

The settings (configs)

Six YAML files under configs/, one per (algorithm × reward weight) combination. They set the sharing weight, network sizes, learning rate, batch sizes, etc. They are identical except for the two things being studied — the algorithm and the reward weight — so the comparison is controlled.

File	What it represents
------	--------------------

harvest_mappo_individual.yaml	MAPPO, w = 0.0
harvest_mappo_mixed.yaml	MAPPO, w = 0.5
harvest_mappo_team.yaml	MAPPO, w = 1.0
harvest_ippo_individual.yaml	IPPO, w = 0.0
harvest_ippo_mixed.yaml	IPPO, w = 0.5
harvest_ippo_team.yaml	IPPO, w = 1.0

Training, orchestration, and analysis

File	What it represents
train_mappo_harvest.py	Trains one MAPPO run from a config. Sets up RLlib/PPO, wires in the centralized-critic callback, runs the iterations, and saves checkpoints.
train_ippo_harvest.py	Same, for one IPPO run (independent critics; no global-state callback needed beyond the shared time feature).
run_matrix.py	The driver that runs the whole 30-run RL matrix one after another. It is idempotent: it stamps each finished run and skips it on restart, so the multi-day job can be stopped and resumed safely. It also tolerates a known harmless Windows crash by checking for a saved checkpoint instead of the exit code.
run_fixed_greedy.py	Runs the hand-written FixedGreedy baseline (the 3 non-learning runs).
evaluate_harvest.py	Loads a trained checkpoint (MAPPO or IPPO) and plays it deterministically for several episodes, writing out the four metrics per episode.
plot_harvest_eval.py	Turns the evaluation CSVs into figures: the apples-on-grid curve (the commons trajectory), bar charts of the four metrics, and per-agent apple counts (to spot one agent hoarding).
tests/	Smoke tests: that the env produces correct shapes, the networks run, the configs all build, and the centralized-critic data path works end to end.

7. How a single run flows, end to end

1. **Config** (e.g. `harvest_mappo_team.yaml`) sets the reward weight and hyperparameters. 2. `run_matrix.py` launches `train*_harvest.py`, which trains for 200 iterations and writes **checkpoints** to `results/`. 3. `evaluate_harvest.py` replays each checkpoint and emits **metric CSVs**. 4. `plot_harvest_eval.py` turns those into **figures**. 5. Finally, the Phase-2 numbers are compared against Phase-1 (traffic) for the cross-environment synthesis — the heart of the thesis.

8. Current status (and the bug that was just fixed)

An earlier attempt at the matrix produced results that had to be discarded because of two bugs, now fixed and verified:

- **The centralized critic was secretly blind.** Due to a quirk of the RLlib library version, the code that was supposed to feed the whole grid into MAPPO's critic never actually ran — so MAPPO was training without its defining feature. It is now rewired correctly and confirmed (the grid data really reaches the critic in a live training step).
- **The critic's learning signal was clipped to near-zero.** A capping value meant for small rewards was throttling Harvest's much larger returns. Raised across all six configs.

All checks now pass — environment shapes, network forward passes, every config builds, and the critical data path is verified end to end. **Phase 2 is ready to run.** The next step is to launch the matrix (canarying the first finished run to confirm the critic is genuinely learning before committing the full ~4 days), then run the FixedGreedy baseline, evaluate, and plot.

In one line: Harvest is a shared apple field where greed destroys the commons; we are testing whether sharing rewards and/or using a coordinated algorithm teaches four agents to harvest sustainably instead — and contrasting that with the traffic phase, where cooperation was free.